

递归例题-整数拆分

题目：一个正整数可以拆分成若干个正整数的和，如 $5=1+2+2$ 。输出一个整数的所有拆分组合。

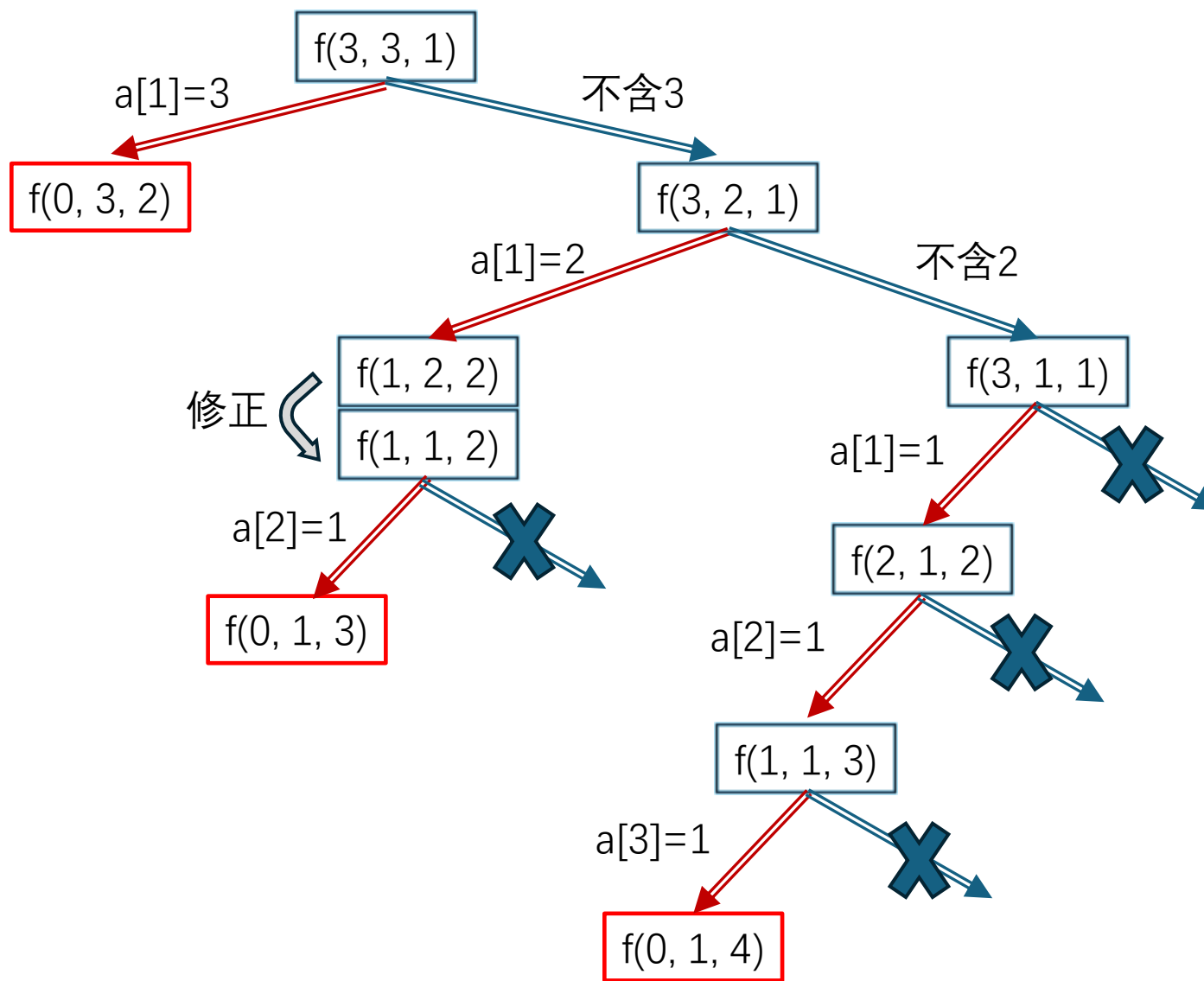
定义：设函数 $f(n,m,i)$ 表示输出整数 n 拆分成不大于 m 的所有组合， i 代表一个拆分组合的第 i 个正整数， $a[i]$ 记录这个正整数。

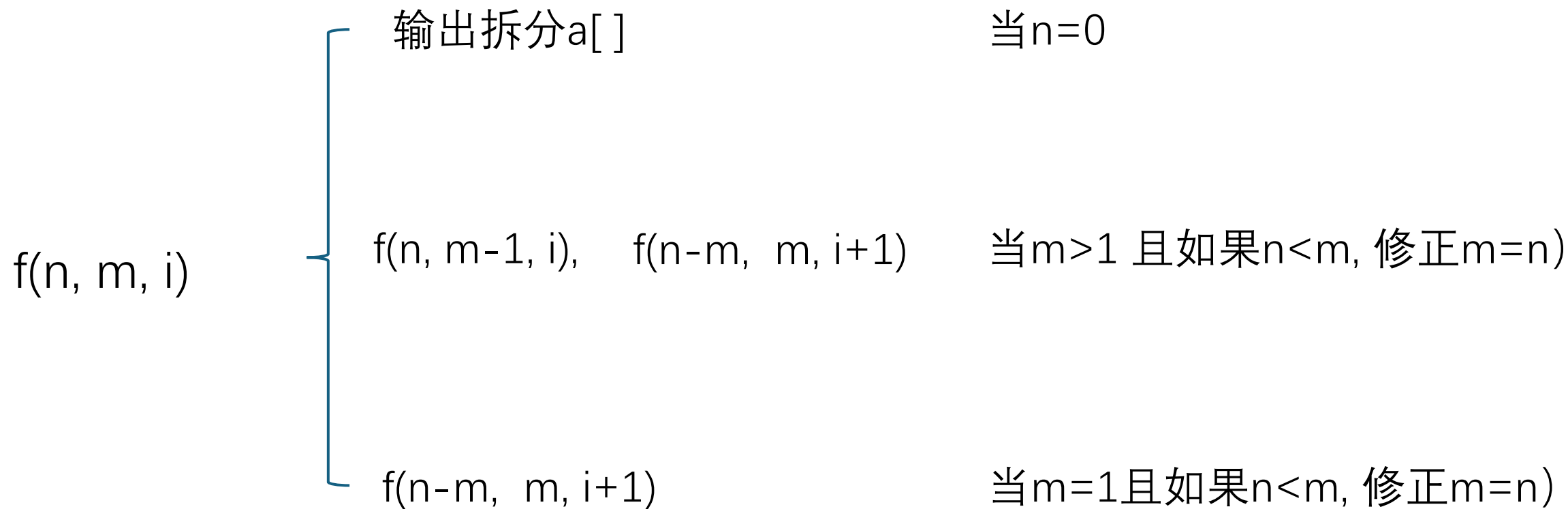
思路：分治思想是把一个复杂的问题分成两个或更多的相同或相似的子问题。

本题可以分成1) 含有至少一个 m 的组合。2) 不含 m 的组合，然后利用递归不断分成两个更小的问题直到直接可解。

举例： $f(3,3,1)$ 拆分成1) 不含“3”的组合 $f(3,2,1)$ 和 2) 含有“3”的组合 $f(0,3,2)$ ，继续这一过程。

递归过程见下图





```
void f(int n, int m, int i)
{
    if(n==0){
        puta(i); //输出数组a[1]..[i], int a[N];
        return;
    }
    if(n<m) m = n; //合法性调整
    //不含m
    if(m>1) f(n, m-1, i);
    //含m
    a[i] = m;
    f(n-m, m, i+1);
}
```

递归和深搜——整数排列

题目：输出由数字1到N（ $N \leq 9$ ）组成的所有N位数字排列，每个数字只出现一次。

定义：设函数 $f(n, i)$ 表示输出n位数字组合，i代表一个枚举第i个正整数， $a[i]$ 记录这个正整数。
 $his[i]$ 表示数字i是否已用。

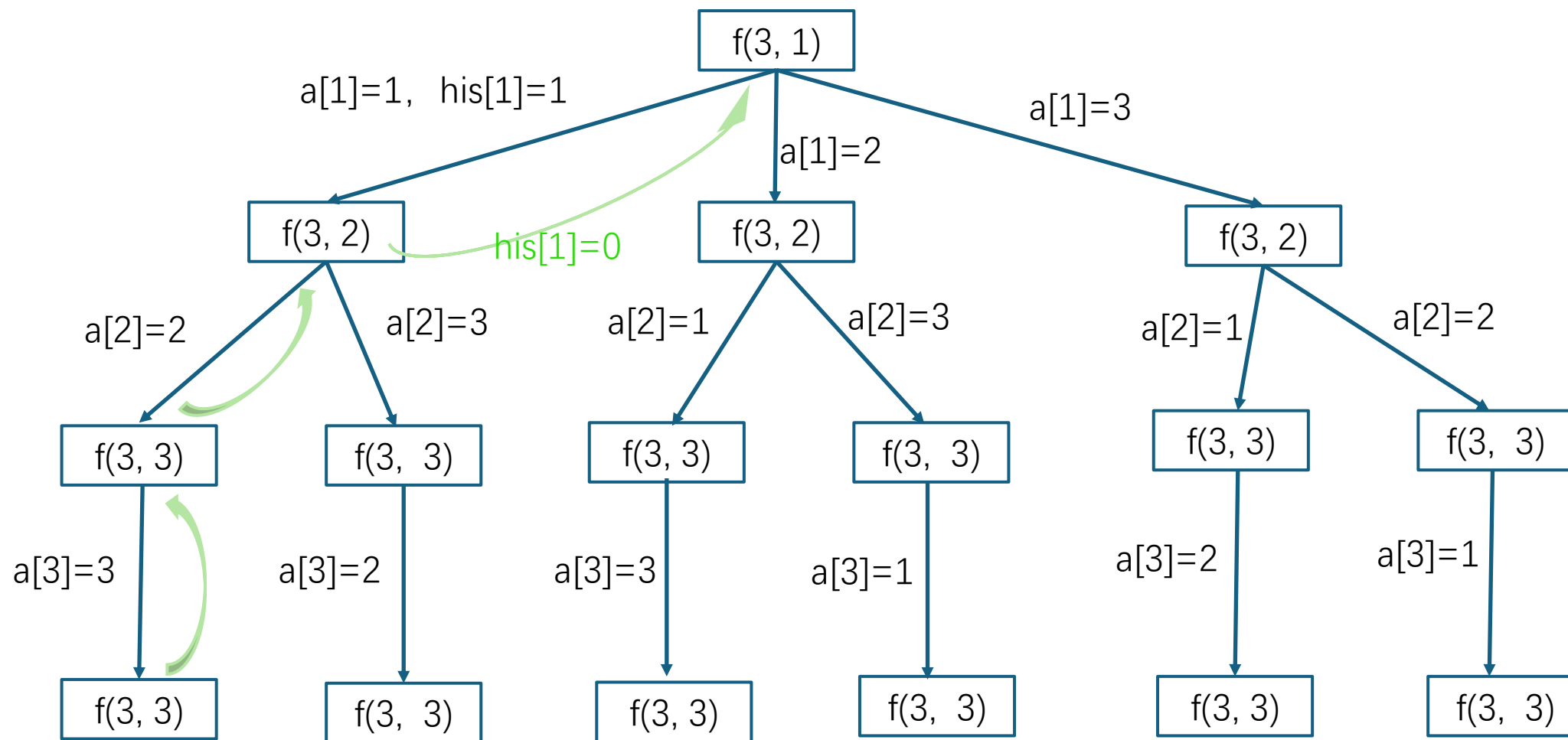
思路：用深搜的思想，将一个排列分成n步，每一步最多有n个选择，每次选择需判断该数字是否已用。

$a[i]$

1	2	3	4	5
---	---	---	---	---

深搜一般框架

```
func(..)
{
    if (目标出现) 终止递归 return
    for(所有选择){
        if(当前选择合法)
        {
            保存选择
            func(...); //递归调用
            复位选择
        }
    }
}
```



```
int a[N];
int his[N];

void f(int n, int i)
{
    if(i>n){
        puta(n); //输出a[1]~a[n]
        return;
    }

    for(int j=1; j<=n; j++){
        if(his[j]==0){ //数字j未使用过
            a[i] = j;
            his[j] = 1; //保存已使用
            f(n, i+1); //递归搜索
            his[j] = 0; //复位释放数字j
        }
    }
}
```